AutoTestAI

Final Project — Technology Stack

This document defines the final technology stack for the AutoTestAI final-year project, covering the implemented foundation and the technologies planned for the remaining development phase.

1. Frontend / User Interface

Technology	Purpose
HTML5	Basic web structure
CSS3	Styling and layout
JavaScript	Frontend logic and API communication
React.js	Interactive dashboard and user interface
Axios / Fetch API	Frontend-backend communication

2. Backend

Technology	Purpose
Python 3.12	Primary programming language
FastAPI	REST API and backend framework
Uvicorn	ASGI server for FastAPI

3. Database

Technology	Purpose
PostgreSQL	Primary relational database
SQLAlchemy	ORM for database communication
Alembic	Database migration management

4. Authentication & Security

Technology	Purpose
JWT	Token-based authentication
OAuth2PasswordBearer	FastAPI bearer-token authentication
Password Hashing	Secure password storage and verification

5. Video Processing

Technology	Purpose
OpenCV (cv2)	Reading screen recordings and extracting frames

6. Image Processing

Technology	Purpose
Pillow	Image loading, resizing, compression and preprocessing

7. OCR / Text Recognition

Technology	Purpose
Tesseract OCR	Extracting exact visible text from frames
pytesseract	Python interface for Tesseract

8. Vision AI

Technology	Purpose
Ollama	Local AI model runtime
LLaVA 7B	Visual understanding of screenshots and UI context

9. Generative AI / AI Analysis

Technology	Purpose
Multimodal AI pipeline	Combines OCR text, coordinates, frame information and visual AI analysis
Test-case generation	Converts observed UI actions/context into structured software test cases

10. API Testing & Documentation

Technology	Purpose
FastAPI Swagger UI	Interactive API documentation and endpoint testing
Postman	API testing and validation

11. Development Tools

Technology	Purpose
Visual Studio Code	Development environment
Git	Version control
GitHub	Source-code repository and collaboration
PowerShell	Windows command-line environment
Python Virtual Environment (venv)	Isolated Python dependency environment

12. AutoTestAI System Architecture

Screen Recording → OpenCV → Frame Extraction → OCR + LLaVA → AI Analysis → Action Recognition → Test Case Generation → Database / Dashboard

Multimodal Analysis Pipeline

Stage	Technology	Role
Screen Recording	Video input	Source of user interaction
Frame Extraction	OpenCV	Extract meaningful frames
Text Extraction	Tesseract OCR	Read visible text and UI labels
Visual Understanding	Ollama + LLaVA 7B	Understand visual context and UI state
Image Preprocessing	Pillow	Resize, compress and prepare images
AI Analysis	Multimodal pipeline	Combine OCR and visual information
Test Generation	Generative AI	Produce structured test cases
Persistence	PostgreSQL + SQLAlchemy	Store project, video, analysis and test-case data
API	FastAPI	Expose project functionality through REST endpoints

13. Current Implementation Status

Status	**Items**
Implemented	FastAPI, PostgreSQL, SQLAlchemy, Alembic, JWT authentication, project CRUD, video upload, OpenCV frame extraction, Ollama, LLaVA and Tesseract OCR.
Next Development	OCR coordinates, frame-analysis service, action detection, test-case generation, test-case database, test execution/validation and frontend dashboard.

Scope note: The stack is intentionally focused for the October 15 project deadline. Items listed as next development should be treated as planned until implemented and tested.